

VESTE.AI

Documentação completa do MVP - plataforma de recomendação de tamanho e estimativa de caimento.

TCC Engenharia de Dados · FIAP · 2026 · Allan · Jhony · Aécio

Documentação VESTE.AI — Índice

Este conjunto descreve o MVP da plataforma: o que ela faz, como está organizada, como o motor calcula, como a API e o widget se usam, e como reproduzir a demonstração.

Cada arquivo também é gerado em PDF em docs/pdf/ pelo comando `npm run docs:pdf`.

#	Documento	Para quem	Como usar
01	[Visão geral](01-visao-geral.md)	Banca e equipe	Entender o problema e a mensagem central
02	[Arquitetura](02-arquitetura.md)	Engenharia	Ver as camadas e o fluxo de dados
03	[Motor de recomendação](03-motor-recomendacao.md)	Engenharia / banca	Explicar o score
04	[API](04-api.md)	Integração B2B	Consumir o endpoint
05	[Experiência do consumidor](05-experiencia-consumidor.md)	Demo	Percorrer as telas
06	[Studio B2B](06-studio-b2b.md)	Demo empresa	Cadastrar produto e testar API
07	[Widget](07-widget.md)	Parceiro	Incorporar ` <code><VesteFit /></code> `
08	[Banco e seed](08-banco-e-seed.md)	Dados	Popular e resetar o ambiente
09	[Como usar](09-como-usar.md)	Operação	Subir local ou Docker
10	[Pipelines e testes](10-pipelines-e-testes.md)	QA	Rodar exemplos e regressão
11	[Limitações e próximos passos](11-limitacoes-e-proximos-passos.md)	Banca	Deixar claro o recorte do MVP

Leitura mínima para a apresentação: 01 -> 03 -> 09 -> 07.

01 — Visão geral do produto

Problema

Uma foto bem produzida da roupa não garante que o consumidor compre o tamanho ou a modelagem mais compatível. A decisão hoje mistura tentativa, tabela genérica e devolução.

Proposta

A VESTE.AI combina:

- dados do usuário (altura, peso, medidas, preferência de caimento);
- dados técnicos da peça (SKU, medidas, modelagem, tecido, elasticidade);
- opcionalmente, proporções estimadas a partir de uma foto autorizada;

para devolver:

- tamanho recomendado;
- score de caimento de 0 a 10;
- nível de confiança;
- diferenças por região;
- justificativa explicável;
- sugestão de ajuste.

Dois públicos

1. Consumidor final - cria perfil, navega no catálogo e analisa o caimento.
2. Empresa / e-commerce - cadastra produtos e consome o mesmo motor por API ou widget.

Mensagem central

A VESTE.AI não diz se uma roupa fica bonita ou feia em uma pessoa. Ela usa dados para estimar como aquela peça tende a vestir e ajudar o consumidor a tomar uma decisão de compra mais informada.

O score nunca é uma nota para o corpo.

O que a banca deve entender

1. Qual problema é resolvido.
2. Quais dados alimentam a decisão.
3. Como a recomendação é calculada e explicada.
4. Como o MVP acadêmico vira produto integrado a e-commerces.

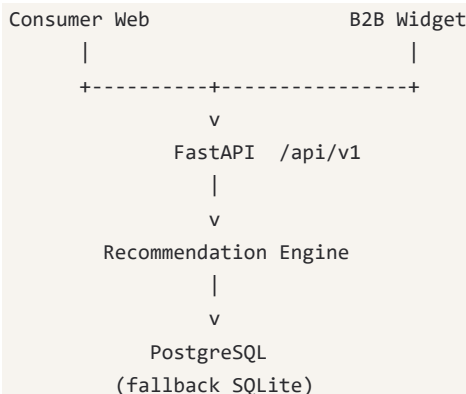
Recorte consciente

Não há app nativo, marketplace, pagamentos, rede social, provador 3D nem modelo de ML em larga escala. O foco é provar viabilidade técnica: persistência, motor real, API e widget no mesmo núcleo.

02 — Arquitetura

Princípio

Não existe front-end fake. A UI consulta a API, a API consulta o banco, o banco tem seed, o motor calcula, e o widget chama o mesmo endpoint.



Monorepo

```

veste-ai/
+-- apps/web/           Next.js - consumidor, studio, loja parceira
+-- apps/api/           FastAPI - HTTP + serviços + motor
|   +-- app/engine/     Núcleo determinístico (sem HTTP)
+-- packages/widget/    <VesteFit />
+-- packages/contracts/ Tipos TypeScript do contrato
+-- packages/ui/        Logo e tokens
+-- database/seed/      Catálogo JSON
+-- docker/             Compose + Dockerfiles
+-- pipelines/          Exemplos e testes
+-- docs/
  
```

O motor fica separado da camada HTTP. Rotas chamam services; services chamam RecommendationEngine.

Camadas da API

```

API
+-- Services
|   +-- Recommendation Engine
|       +-- Measurement Compatibility
|       +-- Fit Preference
|       +-- Modeling Tolerance
|       +-- Fabric Elasticity
|       +-- Visual Proportion
|       +-- Score
|       +-- Confidence
  
```

Persistência

Entidades: User, UserMeasurement, PhotoAnalysis, Product, SKUSize, GarmentMeasurement, FitAnalysis, Feedback, Company, IntegrationKey.

Cada análise guarda usuário (quando houver), SKU, tamanho avaliado, score, confiança, diferenças por região e data/hora.

Canais

Canal	Origem	Header
`web`	Aplicação própria	opcional
`api`	Tester / parceiro	`X-API-Key`
`widget`	Loja parceira	`X-API-Key`

Os três canais executam o mesmo `engine.recommend()`.

Visão computacional

Módulo experimental. Se MediaPipe/OpenCV não estiverem instalados, um fallback heurístico (ou a indisponibilidade explícita) não quebra o fluxo. Sem foto, o peso visual é redistribuído e a confiança cai.

Offline

A banca não depende de APIs externas de IA. Seed + SQLite bastam para a demonstração.

03 — Motor de recomendação

Implementação: apps/api/app/engine/.

O motor é determinístico: mesmas entradas produzem a mesma saída. Não há chamada a modelo generativo.

Fórmula oficial do MVP

```
score = 0.40 × measurements
      + 0.20 × modeling
      + 0.15 × elasticity
      + 0.15 × visual_proportion
      + 0.10 × preference
```

Cada componente sai normalizado em 0-1. O score final é multiplicado por 10 e limitado a 0-10.

Sem foto utilizável, o peso de 15% das proporções é redistribuído proporcionalmente entre os demais.

Componentes

Medidas (40%)

Compara a folga real (medida da peça – medida do corpo) com a folga de projeto da categoria/modelagem. O desvio entra numa curva gaussiana cuja largura é a tolerância da região (peito, cintura, quadril, ombro, comprimento).

Ficar mais justo do que o desenhado pesa um pouco mais do que sobrar ($tight_side_factor = 0.8$).

Modelagem (20%)

Slim, regular, relaxed e oversized absorvem faixas diferentes de desvio (3,5 a 10 cm). O componente mede quanto do erro cabe nessa faixa.

Elasticidade (15%)

Usa o percentual de elasticidade cadastrado na peça. Tecido mais elástico tolera ajuste mais justo.

Proporções visuais (15%, opcional)

A foto não classifica aparência nem faz reconhecimento facial. Extrai relações aproximadas (ombro/quadril, cintura/quadril, tronco/pernas) para ajustar ênfase regional. Sem consentimento ou sem qualidade, o componente é omitido.

Preferência (10%)

Justo, regular ou solto desloca a folga desejada (−3 cm, 0, +4 cm).

Escala de caimento

Faixa	Interpretação	Mensagem
8,5–10	Alta compatibilidade	A peça tende a apresentar bom ajuste nas medidas informadas.
7,0–8,4	Boa compatibilidade	
5,0–6,9	Compatibilidade moderada	O tamanho pode servir, mas recomendamos observar algumas regiões.
0–4,9	Baixa compatibilidade	Outro tamanho ou modelagem tende a oferecer ajuste mais próximo da preferência informada.

Confiança

Nível	Quando
Alta	Medidas completas + análise visual utilizável
Média	Medidas completas, sem foto
Baixa	Medidas incompletas ou cobertura técnica insuficiente

A confiança também considera a margem entre o 1º e o 2º tamanho e a cobertura da ficha técnica.

Regiões

Cada região recebe um status:

- Compatível
- Atenção
- Folga recomendada
- Não avaliado (quando a medida da pessoa ou da peça falta)

Categorias usam regiões diferentes: calça avalia cintura, quadril e comprimento; camiseta inclui peito e ombro.

Comparação entre tamanhos

O motor avalia todos os SKUs da peça e devolve a curva (S/M/L/XL ou numérica). O tamanho recomendado é o de maior score; empate resolve pelo menor desvio médio.

Selecionar outro SKU muda o fit_score do tamanho avaliado, mas o recommended_size permanece o melhor.

Parâmetros

Todos os valores em engine/config.py são heurísticos do MVP. Não devem ser apresentados como cientificamente validados. Podem ser sobrescritos por JSON em VESTE_ENGINE_CONFIG.

Como explicar na banca

1. Mostrar a fórmula.
2. Rodar o perfil demo na camiseta -> M ~ 8,7.
3. Trocar para L e apontar a mudança do score.
4. Abrir "Como calculamos" e o JSON da API.
5. Repetir o mesmo request pelo widget.

04 — API REST v1

Base: `http://localhost:8000`

Prefixo: `/api/v1`

Documentação interativa: `http://localhost:8000/docs`

Especificação: `http://localhost:8000/api/v1/openapi.json`

Autenticação B2B

Endpoints de empresa e cadastro de produto aceitam:

```
X-API-Key: veste_demo_key_loja_parceira
```

A recomendação funciona sem chave (canal web). Com chave, a análise é atribuída à empresa e ao canal (api ou widget).

Endpoints

Saúde

```
GET /api/v1/health
```

Retorna status da API, tipo de banco, modo de visão e contagens.

Consumidor

```
POST /api/v1/users
GET /api/v1/users/{user_id}
PATCH /api/v1/users/{user_id}
PUT /api/v1/users/{user_id}/measurements
POST /api/v1/users/{user_id}/photo
```

O upload de foto exige `consent=true`. A imagem é processada em memória e descartada; só proporções aproximadas são persistidas.

Catálogo

```
GET /api/v1/products
GET /api/v1/products/{id}
GET /api/v1/products/{id}/sizes
POST /api/v1/products
```

`{id}` aceita UUID ou slug.

Motor

```
POST /api/v1/recommendations
GET /api/v1/recommendations/{analysis_id}
POST /api/v1/feedback
GET /api/v1/engine/config
```

Empresa e métricas

```
GET /api/v1/companies/me
GET /api/v1/companies/{company_id}
GET /api/v1/companies/{company_id}/dashboard
GET /api/v1/metrics/dashboard
```

Request de recomendação

Campos aceitos:

- sku ou product_id (+ size opcional)
- customer ou user_id
- fit_preference: tight | regular | loose
- photo_analysis_id, use_photo
- channel: web | api | widget
- persist (padrão verdadeiro)

Exemplo oficial da demonstração:

```
{
  "sku": "CAMISETA-001-M",
  "customer": {
    "height": 180,
    "chest": 102,
    "waist": 88,
    "hip": 100,
    "shoulder": 45
  },
  "fit_preference": "regular"
}
```

Response

```
{
  "recommended_size": "M",
  "fit_score": 8.7,
  "confidence": "medium",
  "regional_analysis": {
    "chest": "good",
    "waist": "good",
    "hip": "attention",
    "shoulder": "good"
  },
  "explanation": "A modelagem apresenta boa compatibilidade com as medidas informadas.",
  "recommendation": "O tamanho M apresenta o melhor equilíbrio entre as regiões analisadas.",
  "comparison": [
    { "size": "S", "fit_score": 6.8, "recommended": false },
    { "size": "M", "fit_score": 8.7, "recommended": true },
    { "size": "L", "fit_score": 7.9, "recommended": false },
    { "size": "XL", "fit_score": 6.9, "recommended": false }
  ]
}
```

A resposta real inclui ainda analysis_id, componentes, pesos, detalhes regionais, mensagens de escala e notas.

cURL

```
curl -X POST http://localhost:8000/api/v1/recommendations \
  -H "Content-Type: application/json" \
  -H "X-API-Key: veste_demo_key_loja_parceira" \
  -d
{"sku":"CAMISETA-001-M","customer":{"height":180,"chest":102,"waist":88,"hip":100,"shoulder":45},"fit_preference":"regular"}
```

Como usar no Studio

Abra `/empresa/api`, edite o JSON e clique em Enviar request. A tela mostra HTTP status, tempo, visualização e o JSON real do backend.

Contratos TypeScript

Os tipos em `packages/contracts/src/index.ts` espelham os schemas Pydantic. Web e widget compartilham o mesmo contrato.

05 — Experiência do consumidor

Rotas Next.js em `apps/web/src/app`.

Landing `/`

Duas portas: Sou consumidor e Sou empresa.

Mensagem comercial:

A VESTE.AI usa dados para estimar como uma peça tende a vestir e ajudar você a escolher um tamanho com mais segurança.

Não há linguagem que avalie o corpo como bonito, feio, ideal ou inadequado.

Cadastro `/consumidor/perfil`

Campos persistidos:

- nome
- altura, peso
- peito/tórax, cintura, quadril, ombro
- preferência: Justo / Regular / Solto
- consentimento explícito para análise da foto

O usuário pode seguir sem foto.

Upload `/consumidor/foto`

- arrastar e soltar
- selecionar arquivo
- preview
- instruções de enquadramento
- aviso: "Imagem utilizada apenas como enriquecimento da estimativa."
- botão Continuar sem foto

A aplicação nunca classifica aparência. Se a visão falhar, o fluxo segue só com medidas e a confiança cai.

Catálogo `/catalogo`

Oito a dez produtos seed (camiseta, camisa, calça, vestido, jaqueta, polo, moletom, shorts). Visual de e-commerce, não de tabela administrativa. Cada card leva ao detalhe.

Produto `/catalogo/[slug]`

Mostra imagem, marca fictícia, modelagem, tecido, elasticidade, composição, tabela técnica por tamanho e o botão Analisar caimento.

Resultado `/analise/[id]`

Tela principal da banca:

- tamanho recomendado
- score 0-10
- confiança
- explicação
- regiões (peito, cintura, quadril, ombros, comprimento)
- comparação S / M / L / XL (ou grade numérica)
- bloco Como calculamos?

Trocar o tamanho reconsulta a API com outro SKU. O score muda de verdade.

Como calculamos `/como-calculamos`

Página permanente com pesos, escala, confiança e parâmetros públicos do motor (GET /api/v1/engine/config).

Como usar na demo

1. Landing -> consumidor.
2. Preencha o perfil demo (180 / 102 / 88 / 100 / 45) ou aceite os valores sugeridos.
3. Pule a foto para ganhar tempo - ou envie uma imagem para mostrar confiança alta.
4. Abra a Camiseta Essencial.
5. Analise e compare M com L.

06 — VESTE.AI Studio (B2B)

Área da empresa em /empresa. Layout próprio (StudioShell) com navegação:

- Visão geral
- Produtos & SKUs
- Testar API
- Widget
- Métricas
- Integrações

Visão geral `/empresa`

Painel com produtos, SKUs, análises, chamadas via API e via widget. Status do banco e da visão computacional. Atalhos para as demais telas.

Produtos `/empresa/produtos`

Lista o catálogo com ficha técnica expansível. Ações:

- ver na loja VESTE.AI
- ver na loja parceira (widget)
- testar via API

Cadastro em /empresa/produtos/novo - nome, categoria, modelagem, tecido, elasticidade e medidas por tamanho. O POST vai para /api/v1/products com a chave demo.

Testar API `/empresa/api`

Request editável + cURL + response visual e JSON. É o mesmo POST /api/v1/recommendations da aplicação e do widget.

Query ?sku=CAMISETA-001-M pré-carrega o payload.

Widget `/empresa/widget`

Pré-visualização ao vivo de <VesteFit /> e snippet de integração.

Métricas `/empresa/metricas`

Gráficos calculados com Pandas sobre análises persistidas:

- análises realizadas
- taxa de recomendação (score $\geq 7,0$)
- tamanho mais recomendado
- score médio e confiança média
- distribuição de tamanhos, faixas de score, categorias e canais
- potencial ilustrativo de redução de devoluções

Sempre identificado como Dados simulados para demonstração.

Integrações `/empresa/integracoes`

Chaves da empresa demo, guia de integração e cURL. Header obrigatório: X-API-Key.

Como usar

Entre por Sou empresa na landing. Mostre produtos -> API -> widget -> métricas. Feche ligando o resultado à hipótese de reduzir devoluções.

07 — Widget incorporável

Pacote: @veste-ai/widget

Componente:

```
import { VestFit } from "@veste-ai/widget";

<VestFit
  productId="camiseta-essential-algodao"
  sku="CAMISETA-001-M"
  apiUrl="http://localhost:8000"
  apiKey="veste_demo_key_loja_parceira"
  label="Descubra seu tamanho ideal"
/>
```

O que o widget faz

1. Mostra o botão Descubra seu tamanho ideal.
 2. Abre um modal com medidas e preferência.
 3. Chama POST /api/v1/recommendations com channel: "widget".
 4. Exibe tamanho, score, confiança, regiões e comparação.
 5. Permite escolher o tamanho para o fluxo da loja (onSelectSize).
- Nenhum cálculo de caimento vive no widget. Ele é um cliente HTTP.

Props

Prop	Uso
`productId`	Id ou slug do produto
`sku`	SKU específico (tem prioridade)
`apiBaseUrl`	URL da API
`apiKey`	Chave `X-API-Key`
`initialMeasurements`	Perfil autorizado / demo
`initialPreference`	`tight` / `regular` / `loose`
`accentColor`	Cor da loja hospedeira
`label`	Texto do botão
`onSelectSize`	Callback da loja
`onResult`	Callback com o JSON

O canal do widget não envia foto. A análise visual fica no fluxo da aplicação VESTE.AI, com consentimento.

Loja parceira (demonstração)

Marca fictícia ATELIER NORTE.

- /loja-parceira - vitrine
- /loja-parceira/produto/[slug] - PDP com o widget

Esta tela prova visualmente que o mesmo motor serve um e-commerce de terceiros.

Build do pacote

```
npm run build:widget
```

Gera ESM e IIFE em packages/widget/dist (script de embed).

Como usar na banca

1. Abra /empresa/widget e dispare o modal.
2. Depois abra /loja-parceira/produto/camiseta-essencial-algodao.
3. Use as medidas já preenchidas (perfil demo) e mostre o resultado.
4. Volte ao Studio e aponte o incremento de "chamadas via widget".

08 — Banco de dados e seed

Banco oficial

PostgreSQL. URL padrão:

```
postgresql+psycopg://veste:veste@localhost:5432/veste_ai
```

Variável: VESTE_DATABASE_URL.

Fallback

Se a conexão falhar e o ambiente não for production, a API usa SQLite local (apps/api/veste_ai.db). Testes automatizados sempre usam SQLite isolado.

Schema

Migração Alembic: apps/api/alembic/versions/0001_initial_schema.py.

No modo demo, VESTE_AUTO_CREATE_SCHEMA=true cria as tabelas na subida.

Entidades

Consumidor: User, UserMeasurement, PhotoAnalysis

Produto: Product, SKUSize, GarmentMeasurement

Recomendação: FitAnalysis, Feedback

B2B: Company, IntegrationKey

FitAnalysis registra usuário (opcional), empresa (opcional), SKU, tamanho avaliado, score, confiança, diferenças por região, canal e data/hora. A foto original não é armazenada.

Seed

Código: apps/api/app/seed/run.py

Catálogo: apps/api/app/seed/catalog.py e database/seed/catalog.json

```
npm run seed
cd apps/api && python -m app.seed.run --reset
```

O que entra:

Item	Conteúdo
Empresa	Loja Parceira Demo ('loja-parceira')
Chaves	'veste_demo_key_loja_parceira' e 'veste_sandbox_key_loja_parceira'
Usuário	Perfil Demonstração — 180 / 78 / 102 / 88 / 100 / 45
Catálogo	10 produtos, grades S–XL ou 38–46
Análises	≈ 320 corridas do próprio motor em perfis sintéticos
Feedback	Amostra simulada para métricas de devolução

Na primeira subida com banco vazio, VESTE_AUTO_SEED=true executa o seed automaticamente.

Produtos do catálogo

Camiseta essencial, camiseta oversized, camisa oxford, calça alfaiataria, calça jeans, vestido midi, jaqueta bomber, polo piquet, moletom canguru, shorts sarja.

Cada SKU tem medidas de peito, cintura, quadril, ombro, comprimento, manga e largura quando aplicável.

Como usar

1. Suba a API sem Postgres para ver o fallback SQLite.
2. Ou use Docker para mostrar PostgreSQL de verdade.
3. Chame GET /api/v1/health e aponte database e products.

09 — Como usar (operação)

Opção A — Local rápido (recomendada para a banca)

1. Instale Node 20+ e Python 3.11+.
2. No diretório veste-ai:

```
npm install
```

3. Em outro terminal, API:

```
cd apps/api
python -m venv .venv
.venv\Scripts\activate
pip install -e ".[dev]"
python -m uvicorn app.main:app --reload --port 8000
```

4. Front-end:

```
npm run dev:web
```

5. Abra <http://localhost:3000> e <http://localhost:8000/docs>.
Sem PostgreSQL a API cria SQLite e popula o seed sozinha.

Opção B — Docker

```
npm run docker:up
```

Serviços:

Serviço	Porta
Web	3000
API	8000
Postgres	5432

Encerrar e apagar o volume:

```
npm run docker:down
```

Variáveis

Copie .env.example. As da API usam prefixo VESTE_.

Variável	Função
`VESTE_DATABASE_URL`	Conexão SQLAlchemy
`VESTE_AUTO_SEED`	Seed se o banco estiver vazio
`VESTE_DEMO_API_KEY`	Chave da empresa demo
`NEXT_PUBLIC_API_URL`	URL da API no navegador
`API_URL`	URL da API no servidor Next (Docker: `http://api:8000`)

Perfil da demonstração

Use estes números para reproduzir o slide da banca:

Campo	Valor
Altura	180 cm

Peso	78 kg
Peito	102 cm
Cintura	88 cm
Quadril	100 cm
Ombro	45 cm
Preferência	Regular
Produto	Camiseta Essencial Algodão
SKU	CAMISETA-001-M

Resultado esperado sem foto: tamanho M, score na faixa 8,5-9,0, confiança média.

Checklist de 5 minutos

1. Landing
2. Consumidor + medidas
3. Foto opcional
4. Catálogo -> camiseta
5. Resultado M
6. Comparar L
7. Como calculamos
8. Studio -> API
9. Loja parceira + widget
10. Métricas (dados simulados)

Problemas comuns

Front sem produtos. A API não está no ar. Confira a porta 8000.

Studio vazio. Mesma causa, ou chave demo diferente da do seed.

Foto falhou. Esperado se a visão não estiver instalada. Siga sem foto.

Porta 3000 ocupada. `npx next dev -p 3001` e ajuste CORS em `VESTE_CORS_ORIGINS`.

10 — Pipelines de exemplos e testes

Há dois pipelines versionados em `pipelines/`.

Pipeline de exemplos

Arquivos JSON em `pipelines/examples/payloads/`. Cada um tem `name`, `request` e `expect`.

Caso	Demonstra
01 camiseta regular	Resultado canônico da banca (M, score alto)
02 camiseta solto	Preferência desloca o equilíbrio
03 calça alfaiataria	Outra categoria e grade numérica
04 medidas incompletas	Confiança baixa
05 corpo menor	Recomendação tende a S/M
06 corpo maior	Recomendação tende a L/XL
07 comparar L	Score muda; recomendado continua M
08 polo widget	Canal widget + API key

Como rodar

```
# Sem servidor - TestClient + SQLite + seed
python pipelines/examples/run_examples.py --inprocess --assert

# API já no ar
python pipelines/examples/run_examples.py --live --base http://localhost:8000

# Atalho npm
npm run pipeline:examples
```

`--assert` faz o processo sair com código 1 se alguma expectativa falhar. Sem a flag, o script só imprime a tabela.

Como adicionar um exemplo

1. Copie um JSON em `payloads/`.
2. Ajuste `request` e `expect`.
3. Rode o pipeline. Não é preciso alterar Python.

Expectativas suportadas: `recommended_size`, `recommended_size_in`, `confidence`, `evaluated_size`, `min_score`, `max_score`.

Pipeline de testes

```
python pipelines/tests/run_all.py
pwsh -File pipelines/tests/run_all.ps1
npm run pipeline:test
```

Ordem:

1. `pytest` em `apps/api` - motor puro + HTTP + persistência
 2. Pipeline de exemplos com `--assert`
 3. Testes Vitest do `@veste-ai/widget`
- Flags: `--skip-widget`, `--skip-examples`.

Testes unitários já existentes

- `apps/api/tests/test_engine.py` - pesos, gaussiana, curva da camiseta, preferência, foto, determinismo, catálogo inteiro

- apps/api/tests/test_api.py - health, OpenAPI, recomendação, usuário, foto, feedback, produto, métricas
- packages/widget/tests/client.test.ts - montagem do request e header
- apps/web/src/lib/utils.test.ts - formatação e faixas de score

```
npm run test:api  
npm run test:widget  
npm run test:web
```

CI

.github/workflows/ci.yml roda pytest, pipeline de exemplos e testes do widget em Ubuntu.

Como usar na banca

Mostre a tabela do pipeline de exemplos no terminal e compare a linha 01 com o JSON da tela /empresa/api. Isso prova que o motor não é um mock.

11 — Limitações do MVP e próximos passos

O que este recorte não é

- Aplicativo mobile nativo
- Marketplace ou rede social
- Proveedor 3D hiper-realista
- Modelo de machine learning treinado em escala
- Integração com dezenas de lojas reais
- Pagamentos ou e-commerce completo

O objetivo é provar viabilidade técnica: dados + motor explicável + API + widget.

Limitações honestas

Parâmetros heurísticos. Pesos, tolerâncias e folgas de projeto não foram validados em estudo clínico ou em produção. Existem para tornar o cálculo transparente e reproduzível.

Visão experimental. A foto é enriquecimento. Sem ela o sistema continua correto, com confiança menor.

Métricas simuladas. O dashboard agrega análises geradas pelo próprio motor em perfis sintéticos. Sempre rotulado como dados de demonstração.

Privacidade no MVP. A imagem não é persistida, mas o ambiente acadêmico não substitui um programa formal de conformidade.

Calibração por marca. Tabelas reais de cada fabricante ainda não entram no aprendizado.

Evolução prevista

1. Coletar feedback real de caimento, compra, troca e devolução.
2. Calibrar o score por marca, categoria, modelagem e tecido.
3. Trocar parte das regras por um modelo supervisionado - mantendo a explicação.
4. Levar o widget a um e-commerce parceiro de verdade.
5. Só então investir em representação visual da peça sobre o usuário, sem abandonar os dados técnicos.

Mensagem para fechar a banca

O valor não é "adivinhar o corpo". É reduzir incerteza na compra com um motor único, auditável, que já serve aplicação própria, API e componente incorporável.